

Ex.No:	
Date:	

AIM:

ALGORITHM:

CODE:

```
def isPrime(n):  
    if n <= 1:  
        return False  
    for i in range(2, int(n**0.5) + 1):  
        if n % i == 0:
```

```
        return False
    return True

p = int(input("Enter the first number (p): "))
q = int(input("Enter the second number (q): "))

if isPrime(p):
    print(p, "is a Prime Number.")
else:
    print(p, "is NOT a Prime Number.")

if isPrime(q):
    print(q, "is a Prime Number.")
else:
    print(q, "is NOT a Prime Number.")

if isPrime(p) and isPrime(q):
    print("\nBoth numbers are prime.")
    print("Proceed with RSA Key Generation.")
```

Output:

```
Enter the first number (p): 17
Enter the second number (q): 23
17 is a Prime Number.
23 is a Prime Number.
```

```
Both numbers are prime.
Proceed with RSA Key Generation.
```

Result:

Ex.No:	
Date:	

AIM:

ALGORITHM:

CODE:
def find_gcd(a,b):
if a<b:
a,b =b,a
print("\nSteps of Euclidean Algorithm.")
while b!=0:
remainder=a%b

```
print(f"{a} ÷ {b} = Quotient {a//b}, Remainder {remainder}")
```

```
a = b
```

```
b = remainder
```

```
return a
```

```
print("=== Secure Communication Key Validation ===")
```

```
parameter1 = int(input("Enter the first encryption parameter: "))
```

```
parameter2 = int(input("Enter the second encryption parameter: "))
```

```
gcd = find_gcd(parameter1, parameter2)
```

```
print("\nGreatest Common Divisor (GCD) =", gcd)
```

```
if gcd == 1:
```

```
    print("\nThe encryption parameters are COPRIME.")
```

```
    print("Secure communication can be established.")
```

```
else:
```

```
    print("\nThe encryption parameters are NOT COPRIME.")
```

```
    print("Communication setup rejected.")
```

```
    print("Choose different encryption parameters.")
```

Output:

```
=== Secure Communication Key Validation ===
```

```
Enter the first encryption parameter: 35
```

```
Enter the second encryption parameter: 12
```

```
Steps of Euclidean Algorithm:
```

```
35 ÷ 12 = Quotient 2, Remainder 11
```

```
12 ÷ 11 = Quotient 1, Remainder 1
```

```
11 ÷ 1 = Quotient 11, Remainder 0
```

```
Greatest Common Divisor (GCD) = 1
```

```
The encryption parameters are COPRIME.
```

```
Secure communication can be established.
```

Result:

Ex.No:	
Date:	

AIM:

ALGORITHM:

CODE:

```
print("=====")
print("    MODULAR ARITHMETIC CALCULATOR")
print("=====")
```

```
a = int(input("Enter the First Integer (a): "))
b = int(input("Enter the Second Integer (b): "))
m = int(input("Enter the Modulus (m): "))
```

```
if m <= 0:
    print("Modulus must be greater than 0.")
```

```
else:
```

```
while True:
    print("\n===== MENU =====")
    print("1. Modular Addition")
    print("2. Modular Subtraction")
    print("3. Modular Multiplication")
    print("4. Modular Exponentiation")
    print("5. Exit")

    choice = int(input("Enter your choice (1-5): "))

    if choice == 1:
        result = (a + b) % m

        print("\nModular Addition")
        print(f"({a} + {b}) mod {m} = {result}")

    elif choice == 2:
        result = (a - b) % m

        print("\nModular Subtraction")
        print(f"({a} - {b}) mod {m} = {result}")

    elif choice == 3:
        result = (a * b) % m

        print("\nModular Multiplication")
        print(f"({a} × {b}) mod {m} = {result}")

    elif choice == 4:
        result = pow(a, b, m)

        print("\nModular Exponentiation")
        print(f"({a}^{b}) mod {m} = {result}")

    elif choice == 5:
        print("\nExiting the Program...")
        break

    else:
        print("Invalid Choice! Please select between 1 and 5.")
```

OUTPUT:

```
=====
MODULAR ARITHMETIC CALCULATOR
=====
Enter the First Integer (a): 15
Enter the Second Integer (b): 7
Enter the Modulus (m): 11

===== MENU =====
1. Modular Addition
2. Modular Subtraction
3. Modular Multiplication
4. Modular Exponentiation
5. Exit
Enter your choice (1-5): 1

Modular Addition
(15 + 7) mod 11 = 0

===== MENU =====
1. Modular Addition
2. Modular Subtraction
3. Modular Multiplication
4. Modular Exponentiation
5. Exit
Enter your choice (1-5): 3

Modular Multiplication
(15 × 7) mod 11 = 6

===== MENU =====
1. Modular Addition
2. Modular Subtraction
3. Modular Multiplication
4. Modular Exponentiation
5. Exit
Enter your choice (1-5): 4

Modular Exponentiation
(15^7) mod 11 = 5

===== MENU =====
1. Modular Addition
2. Modular Subtraction
3. Modular Multiplication
4. Modular Exponentiation
5. Exit
Enter your choice (1-5): 5

Exiting the Program...
```

RESULT:

Ex.No:	
Date:	

AIM:

ALGORITHM:

CODE:

```
print("=====")
print("  RSA Prime Validation using Fermat's Test")
print("=====")
a = int(input("Enter the value of a: "))
p = int(input("Enter the candidate prime
number (p): ")) if p <= 1:
    print("\nInvalid Input!")
print("The candidate number p must be greater than 1.")
```

```

else:
    result = pow(a, p - 1, p)
print("\nResult of Fermat Test") print(f'({a}^{p-
1}) mod {p} = {result}') if result == 1:
    print("\nThe number satisfies Fermat's Little Theorem.") print("It is a
    probable prime.")
    print("Candidate is suitable for further RSA key generation.") else:
    print("\nThe number does not satisfy Fermat's Little Theorem.") print("It is
    composite.")
print("Candidate is NOT suitable for RSA key generation.")

```

OUTPUT:

```

=====
RSA Prime Validation using Fermat's Test
=====
Enter the value of a: 2
Enter the candidate prime number (p): 17

Result of Fermat Test
(2^16) mod 17 = 1

The number satisfies Fermat's Little Theorem.
It is a probable prime.
Candidate is suitable for further RSA key generation.

```

RESULT:

Ex.No:	
Date:	

AIM:

ALGORITHM:

CODE:

```
import math
```

```
a = int(input("Enter the value of a: "))
```

```
n = int(input("Enter the value of n: "))
```

```
phi = 0
```

```
for i in range(1, n + 1):  
    if math.gcd(i, n) == 1:  
        phi += 1
```

```

print("\nEuler's Totient Function")
print("φ(", n, ") =", phi)

if math.gcd(a, n) == 1:
    print(a, "and", n, "are coprime.")

    result = pow(a, phi, n)

    print("\nEuler's Theorem Verification")
    print(f"({a}^{phi}) mod {n} = {result}")

    if result == 1:
        print("Euler's Theorem is Verified.")
    else:
        print("Euler's Theorem is NOT Verified.")
else:
    print(a, "and", n, "are NOT coprime.")
    print("Euler's Theorem cannot be applied.")

```

OUTPUT:

```

Enter the value of a: 3
Enter the value of n: 10

```

```

Euler's Totient Function
φ( 10 ) = 4
3 and 10 are coprime.

```

```

Euler's Theorem Verification
3^4 mod 10 = 1
Euler's Theorem is Verified.

```

RESULT:

Ex.No:	
Date:	

AIM:

ALGORITHM:

CODE:

```
seed = 3
a = 5
c = 2
m = 13
n = 5
sequence=[seed]
x=seed
print("Linear Congruential generator")
print("-----")
```

```
if sequence[-1] in sequence[:-1]:
    print("Sequence Repeats: Yes")
    period = len(sequence) - 1 - sequence.index(sequence[-1])
    print("Period =", period)
else:
    print("Sequence Repeats: No")
```

OUTPUT:

Linear Congruential Generator

X1 = 4
X2 = 9
X3 = 8
X4 = 3
X5 = 4

Generated Sequence:
[3, 4, 9, 8, 3, 4]
Sequence Repeats: Yes
Period = 4

RESULT:

Ex.No:	
Date:	

AIM:

ALGORITHM:

CODE:

```
import math
```

```
p = 7
```

```
q = 11
```

```
seed = 16
```

```
num_bits = 5
```

```
n = p * q
```

```
if math.gcd(seed, n) != 1:  
    print("Invalid Seed")
```

```

else:
    x = seed
    bits = []

    print("Blum Blum Shub Generator")
    print("-----")

    for i in range(num_bits):
        x = (x * x) % n
        bit = x % 2

        bits.append(bit)

        print("Random Number =", x, "Output Bit =", bit)

    print("\nBit Stream =", ".join(map(str, bits)))

```

OUTPUT:

```

Blum Blum Shub Generator
-----
Random Number = 25   Output Bit = 1
Random Number = 9    Output Bit = 1
Random Number = 4    Output Bit = 0
Random Number = 16   Output Bit = 0
Random Number = 25   Output Bit = 1

Bit Stream = 11001

```

RESULT:

Ex.No:	
Date:	

AIM:

ALGORITHM:

CODE:

```
register = [1, 0, 1, 1]
```

```
n = 10
```

```
print("Linear Feedback Shift Register")
```

```
print("-----")
```

```
for i in range(n):
```

```
feedback = register[0] ^ register[3]

output = register[-1]

register = [feedback] + register[:-1]

print("Step", i + 1)

print("Feedback =", feedback)

print("Output =", output)

print("Register =", register)

print()
```

OUTPUT:

Linear Feedback Shift Register

Step 1

Feedback = 0

Output = 1

Register = [0, 1, 0, 1]

Step 2

Feedback = 1

Output = 1

Register = [1, 0, 1, 0]

Step 3

Feedback = 1

Output = 0

Register = [1, 1, 0, 1]

Step 4

Feedback = 0

Output = 1

Register = [0, 1, 1, 0]

Step 5

Feedback = 0

Output = 0

Register = [0, 0, 1, 1]

Step 6
Feedback = 1
Output = 1
Register = [1, 0, 0, 1]

Step 7
Feedback = 0
Output = 1
Register = [0, 1, 0, 0]

Step 8
Feedback = 0
Output = 0
Register = [0, 0, 1, 0]

Step 9
Feedback = 0
Output = 0
Register = [0, 0, 0, 1]

Step 10
Feedback = 1
Output = 1
Register = [1, 0, 0, 0]

RESULT:

Ex.No:	
Date:	

AIM:

ALGORITHM:

CODE:

```
def encrypt(text, shift):
    result = ""
    for ch in text.upper():
        if ch.isalpha():
            result += chr((ord(ch) - 65 + shift) % 26 + 65)
        else:
            result += ch
    return result

def decrypt(text, shift):
    result = ""
    for ch in text:
        if ch.isalpha():
            result += chr((ord(ch) - 65 - shift) % 26 + 65)
        else:
            result += ch
```

```
return result

plain = input("Enter Plaintext : ")
shift = int(input("Enter Key : "))
cipher = encrypt(plain, shift)
print("Ciphertext :", cipher)
original = decrypt(cipher, shift)
print("Decrypted :", original)
```

OUTPUT:

```
Enter Plaintext : HELLO
Enter Key : 3
Ciphertext : KHOOR
Decrypted : HELLO
```

RESULT:

Ex.No:	
Date:	

AIM:

ALGORITHM:

CODE:

```
from string import ascii_uppercase

def generate_key_matrix(key):
    key = key.upper().replace("J", "I")
    matrix = []
    used = set()
    for ch in key + ascii_uppercase:
        if ch == 'J':
            continue
        if ch not in used:
            used.add(ch)
            matrix.append(ch)

    return [matrix[i:i+5] for i in range(0, 25, 5)]
```

```

def find_position(matrix, ch):
    if ch == 'J':
        ch = 'I'
    for i, row in enumerate(matrix):
        if ch in row:
            return i, row.index(ch)

def prepare_text(text):
    text = text.upper().replace("J", "I")
    text = "".join(ch for ch in text if ch.isalpha())
    result = ""
    i = 0
    while i < len(text):
        a = text[i]
        if i + 1 < len(text):
            b = text[i + 1]
            if a == b:
                result += a + "X"
                i += 1
            else:
                result += a + b
                i += 2
        else:
            result += a + "X"
            i += 1
    return result

def encrypt(text, matrix):

```

```
text = prepare_text(text)

cipher = ""

for i in range(0, len(text), 2):

    a = text[i]

    b = text[i + 1]

    r1, c1 = find_position(matrix, a)
    r2, c2 = find_position(matrix, b)

    if r1 == r2:

        cipher += matrix[r1][(c1 + 1) % 5]
        cipher += matrix[r2][(c2 + 1) % 5]

    elif c1 == c2:

        cipher += matrix[(r1 + 1) % 5][c1]
        cipher += matrix[(r2 + 1) % 5][c2]

    else:

        cipher += matrix[r1][c2]
        cipher += matrix[r2][c1]

return cipher

def decrypt(cipher, matrix):

    plain = ""

    for i in range(0, len(cipher), 2):

        a = cipher[i]

        b = cipher[i + 1]

        r1, c1 = find_position(matrix, a)

        r2, c2 = find_position(matrix, b)

        if r1 == r2:
```

```

        plain += matrix[r1][(c1 - 1) % 5]
        plain += matrix[r2][(c2 - 1) % 5]
elif c1 == c2:
    plain += matrix[(r1 - 1) % 5][c1]
    plain += matrix[(r2 - 1) % 5][c2]
else:
    plain += matrix[r1][c2]
    plain += matrix[r2][c1]
cleaned = ""
i = 0
while i < len(plain):
    if (
        i + 2 < len(plain)
        and plain[i] == plain[i + 2]
        and plain[i + 1] == 'X'
    ):
        cleaned += plain[i]
        i += 2
    else:
        cleaned += plain[i]
        i += 1
    if cleaned.endswith('X'):
        cleaned = cleaned[:-1]
return cleaned

key = input("Enter Keyword : ")
plain = input("Enter Plaintext : ")

```

```
matrix = generate_key_matrix(key)
```

```
print("\nKey Matrix")
```

```
for row in matrix:
```

```
    print(" ".join(row))
```

```
cipher = encrypt(plain, matrix)
```

```
print("\nCiphertext :", cipher)
```

```
original = decrypt(cipher, matrix)
```

```
print("Decrypted Text :", original)
```

OUTPUT:

```
Enter Keyword : PLAY
```

```
Enter Plaintext : HELLO
```

```
Key Matrix
```

```
P L A Y B
```

```
C D E F G
```

```
H I K M N
```

```
O Q R S T
```

```
U V W X Z
```

```
Ciphertext : KCYVPQ
```

```
Decrypted Text : HELLO
```

RESULT:

Ex.No:	
Date:	

AIM:

ALGORITHM:

CODE:

```
import numpy as np
key = np.array([[3, 3], [2, 5]])
inverse = np.array([[15, 17], [20, 9]])
def process(text):
    text = text.upper()
    if len(text) % 2 != 0:
        text += "X"
    return text
def convert(text):
    return [ord(i) - 65 for i in text]
def encrypt(text):
    text = process(text)
    nums = convert(text)
    cipher = ""
    for i in range(0, len(nums), 2):
        pair = np.array([[nums[i]], [nums[i + 1]]])
        result = np.dot(key, pair) % 26
```

```
cipher += chr(result[0][0] + 65)
cipher += chr(result[1][0] + 65)
return cipher
def decrypt(cipher):
    nums = convert(cipher)
    plain = ""
    for i in range(0, len(nums), 2):
        pair = np.array([[nums[i]], [nums[i + 1]]])
        result = np.dot(inverse, pair) % 26
        plain += chr(result[0][0] + 65)
        plain += chr(result[1][0] + 65)
    return plain
plain = input("Enter Plaintext : ")
cipher = encrypt(plain)
print("Ciphertext :", cipher)
print("Decrypted :", decrypt(cipher))
```

OUTPUT:

```
Enter Plaintext : HELP
Ciphertext : HIAT
Decrypted : HELP
```

RESULT:

Ex.No:	
Date:	

AIM:

ALGORITHM:

CODE:

```
def generateKey(text, key):
    key = key.upper()
    while len(key) < len(text):
        key += key
    return key[:len(text)]
def encrypt(text, key):
    cipher = ""
    for i in range(len(text)):
        x = (ord(text[i]) - 65 + ord(key[i]) - 65) % 26
        cipher += chr(x + 65)
    return cipher
def decrypt(cipher, key):
    plain = ""
    for i in range(len(cipher)):
```

```
x = (ord(cipher[i]) - 65 - (ord(key[i]) - 65)) % 26
plain += chr(x + 65)
return plain
plain = input("Enter Plaintext : ").upper()
keyword = input("Enter Keyword : ")
key = generateKey(plain, keyword)
cipher = encrypt(plain, key)
print("Generated Key :", key)
print("Ciphertext :", cipher)
original = decrypt(cipher, key)
print("Decrypted :", original)
```

OUTPUT:

```
Enter Plaintext : ATTACKATDAWN
Enter Keyword : LEMON
Generated Key : LEMONLEMONLE
Ciphertext : LXFOPVEFRNHR
Decrypted : ATTACKATDAWN
```

RESULT:

Ex.No:	
Date:	

AIM:

ALGORITHM:

CODE:

```
from Crypto.Cipher import DES
from Crypto.Util.Padding import
pad, unpad key = b'12345678'
cipher = DES.new(key,
DES.MODE_ECB) plaintext =
input("Enter Plaintext: ")
ciphertext = cipher.encrypt(pad(plaintext.encode(), DES.block_size))

print("Encrypted Data:",
```

```
ciphertext.hex()) decipher =  
DES.new(key, DES.MODE_ECB)  
decrypted = unpad(decipher.decrypt(ciphertext),  
DES.block_size) print("Decrypted Data:",  
decrypted.decode())
```

OUTPUT:

Enter Plaintext: HELLO

Encrypted Data: 8b6a14412de58903

Decrypted Data: HELLO

RESULT:

Ex.No:	
Date:	

AIM:

ALGORITHM:

CODE:

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def mod_inverse(e, phi):
    for d in range(1, phi):
        if (e * d) % phi == 1:
            return d
    return None

p = 17
q = 11
n = p * q
phi = (p - 1) * (q - 1)
e = 7
d = mod_inverse(e, phi)
print("Public Key:", (e, n))
print("Private Key:", (d, n))
message = int(input("Enter Message (<187): "))
cipher = pow(message, e, n)
```

```
print("Encrypted Message:", cipher)
plain = pow(cipher, d, n)
print("Decrypted Message:", plain)
```

OUTPUT:

```
Public Key: (7, 187)
Private Key: (23, 187)
Enter Message (<187): 11
Encrypted Message: 88
Decrypted Message: 11
```

RESULT:

Ex.No:	
Date:	

AIM:

ALGORITHM:

CODE:

```
import hashlib  
message = input("Enter Message:  
")  
  
hash_value  
=hashlib.md5(message.encode())  
print("MD5 Digest:",  
hash_value.hexdigest())
```

OUTPUT:

```
Enter Message: Cryptology  
MD5 Digest: c6ae9d0a089311bab119969161ca5cce
```

RESULT:

Ex.No:	
Date:	

AIM:

ALGORITHM:

CODE:

```
import hashlib
message=input("Enter Message: ")
hash_value=hashlib.sha1(message.encode())
print("SHA-1 Digest:",hash_value.hexdigest())
```

OUTPUT:

```
Enter Message: Cryptology
SHA-1 Digest: ba7160026c971c61eca61f590d68b3de9fadc0f3
```

RESULT: